ShanghaiTech University

CS240 Algorithm Design and Analysis

Spring 2025

Problem Set 4

Due: 11:59 pm, May 22, 2025

1. Submit your solutions to the course Gradescope.
2. If you want to submit a handwritten version, scan it clearly.
3. Late submissions are not allowed.
4. You are required to follow ShanghaiTech's academic honesty policies. You are allowed to discuss problems with other students, but you must write up your solutions by yourselves. You are not allowed to copy materials from other students or from online or published resources. Violating academic honesty can result in serious penalties.

Problem 1:

Sometimes, local search algorithms can give good approximations to NP-hard problems. In the Max-Cut problem, we have a graph $G(V, E)$ and we want to find a cut (S, T) with as many edges crossing as possible. One local search algorithm is as follows: Start with any cut, and while there is some vertex $v \in S$ such that more edges cross $(S - v, T + v)$ (or some $v \in T$ such that more edges cross $(S + v, T - v)$), move v to the other side of the cut. Note that when we move v from S to T, v must have more neighbors in S than T.

a Give an upper bound on the number of iterations this algorithm can run for (i.e. the total number of times we move a vertex).

b Show that when this algorithm terminates, it finds a cut where at least half the edges in the graph cross the cut.

Solution:

a $|E|$ iterations. Each iteration increases the number of edges crossing the cut by at least 1. The number of edges crossing the cut is between 0 and $|E|$, so there must be at most $|E|$ iterations.

b $\delta_{in}(v)$ be the number of edges from v to other vertices on the same side of the cut, and $\delta_{\text{out}}(v)$ be the number of edges from v to vertices on the opposite side of the cut. The total number of edges crossing the cut the algorithm finds is $\frac{1}{2}\sum_{v\in V}\delta_{out}(v)$, and the total number of edges in the graph is $\frac{1}{2}\sum_{v\in V}(\delta_{\text{in}}(v)+\delta_{\text{out}}(v))$. We know that $\delta_{\text{out}}(v) \geq \delta_{\text{in}}(v)$ for all vertices when the algorithm terminates (otherwise, the algorithm would move v across the cut), so the former is at least half as large as the latter.

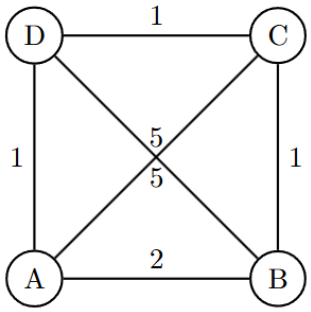
Problem 2:

Consider the following approximation algorithm for TSP:

1. Given the complete graph $G = (V, E)$, compute its MST (Minimum spanning tree).
2. Run DFS on the MST computed in the previous step, and record down the vertices visited in pre-order.
3. The output tour is the list of vertices computed in the previous step, with the first vertex appended to the end.

When G satisfies the triangle inequality, this algorithm achieves an approximation factor of 2. But what happens when triangle inequality does not hold?

Suppose we run this approximation algorithm on the following graph:



The algorithm will return different tours based on the choices it makes during its depth first traversal.

a Which DFS traversal leads to the best possible output tour?

b Which DFS traversal leads to the worst possible output tour?

c What is the approximation ratio given by the algorithm in the worst case for the above instance? Why is it worse than 2?

Solution:

a **A-D-C-B, D-C-B-A, B-C-D-A, or C-D-A-B**
Explanation:
The MST is $\{(A, D), (D, C), (C, B)\}$. The optimal tour uses all of these edges.
For example, if we start traversing the tree at A, then the only traversal would be to follow the tree and go through the vertices in the order D, C, B.
Another potential traversal would be to start at D, move to C, then B, and backtrack, before going to A.
Notice that if we started at D and moved to A first, then we would have to visit C next, and this would not lead to the best possible output tour (as per part (b)).

b **C-B-D-A or D-A-C-B**
See above explanation.
Notice that the worst possible tour overall, **D-C-A-B**, is not possible to be output by this algorithm, regardless of the DFS traversal used.

c $\frac{12}{5}$
From previous parts, the optimal tour length is 5 while the worst possible is 12. This is worse than 2 because our graph does not satisfy the triangle inequality, specifically $\ell(A, C) > \ell(A, B) + \ell(B, C)$ and $\ell(B, D) > \ell(A, B) + \ell(D, A)$, which is the necessary condition for our algorithm to guarantee an approximation ratio of 2.

Problem 3:

A delivery company needs to load n packages into trucks. Each truck has a maximum load capacity of C, and the weight of the n packages is denoted as $\{w_1, w_2, \ldots, w_n\}$, where $w_i \leq C$ $(1 \leq i \leq n)$.

Design a polynomial-time 2-approximation algorithm to solve this problem and prove the correctness of your algorithm.

Solution:

Algorithm: First, initialize all the trucks as empty. Then, take each package one by one and place it into the first truck that can accommodate it.

The basic idea of the algorithm is to find the first truck that can accommodate package i. Its time complexity is $O\left(n^2\right)$.

Without loss of generality, assume the capacity of each truck is C, and C_i represents the total volume of packages packed in the i-th truck. Using the strategy, we have $C_i + C_j > 1$. Suppose the approximate solution for the bin packing problem is k, then:

- If k is even, $C_1 + C_2 + \cdots + C_k > k/2$.
- If k is odd, $C_1 + C_2 + \cdots + C_k > (k-1)/2$.

Adding these two equations, we get:

$$2\sum_{i=1}^{k} C_i > \frac{2k-1}{2}$$

The optimal solution to the bin packing problem is m. At optimal packing, all trucks are fully used to pack all packages, i.e.,

$$m = \sum_{i=1}^{m} C_m = \sum_{i=1}^{n} s_i$$

Thus, we have:

$$2\sum_{i=1}^{k} C_i = 2\sum_{i=1}^{n} s_i = 2m > \frac{2k-1}{2}$$

This implies:

$$\frac{k}{m} < 2$$

Therefore, the approximation ratio of the algorithm is less than 2 .

Problem 4:

In this problem, we will analyze the time complexity of QUICKSORT in terms of error probabilities, rather than in terms of expectation. Suppose the array to be sorted is $A[1 \ldots n]$, and write x_i for the element that starts in array location $A[i]$ (before QUICKSORT is called). Assume that all the x_i values are distinct.

In solving this problem, it will be useful to recall a claim from the lecture. Here it is, slightly restated: Claim: Let $c > 1$ be a real constant, and let α be a positive integer. Then, with probability at least $1 - \frac{1}{n^\alpha}, 3(\alpha + c) \lg n$ tosses of a fair coin produce at least $c \lg n$ heads.

a Consider a particular element x_i. Consider a recursive call of QUICKSORT on subarray $A[p \ldots p + m - 1]$ of size $m \geq 2$ which includes element x_i. Prove that, with probability at least $\frac{1}{2}$, either this call to QUICKSORT chooses x_i as the pivot element, or the next recursive call to QUICKSORT containing x_i involves a subarray of size at most $\frac{3}{4}m$.

b Consider a particular element x_i. Prove that, with probability at least $1 - \frac{1}{n^2}$, the total number of times the algorithm compares x_i with pivots is at most $d \lg n$, for a particular constant d. Give a value for d explicitly.

c Now consider all of the elements $x_1, x_2, \ldots, x_n$. Apply your result from part (b) to prove that, with probability at least $1 - \frac{1}{n}$, the total number of comparisons made by QUICKSORT on the given array input is at most $d' n \lg n$, for a particular constant d'. Give a value for d' explicitly. Hint: The Union Bound may be useful for your analysis.

d Generalize your results above to obtain a bound on the number of comparisons made by QUICKSORT that holds with probability $1 - \frac{1}{n^\alpha}$, for any positive integer α, rather than just probability $1 - \frac{1}{n}$ (i.e., $\alpha = 1$).

Solution:

a Suppose the pivot value is x. If $\left\lfloor \frac{m}{4} \right\rfloor + 1 \leq x \leq m - \left\lfloor \frac{m}{4} \right\rfloor$, then both subarrays produced by the partition have size at most $\frac{3m}{4}$. Moreover, the number of values of x in this range is at least $\frac{m}{2}$, so the probability of choosing such a value is at least $\frac{1}{2}$. Then either x_i is the pivot value or it is in one of the two segments.

b We use part (a) and the Claim. By part (a), each time Quicksort is called for a subarray containing x_i, with probability at least $\frac{1}{2}$, either x_i is chosen as the pivot value or else the size of the subarray containing x_i reduces to at most $\frac{3}{4}$ of what it was before the call. Let's say that a call is "successful" if either of these two cases happens. That is, with probability at least $\frac{1}{2}$, the call is successful. Now, at most $\log_{4/3} n$ successful calls can occur for subarrays containing x_i during an execution, because after that many successful calls, the size of the subarray containing x_i would be reduced to 1 . Using the change of base formula for logarithms, $\log_{4/3} n = c \lg n$, where $c = \log_{4/3} 2$. Now we can model the sequence of calls to QUICKSORT for subarrays containing x_i as a sequence of tosses of a fair coin, where heads corresponds to successful calls. By the Claim, with $c = \log_{4/3} 2$ and $\alpha = 2$, we conclude that, with probability at least $1 - \frac{1}{n^2}$, we have at least $c \lg n$ successful calls within $d \lg n$ total calls, where $d = 3(2 + c)$. Each comparison of x_i with a pivot occurs as part of one of these calls, so with probability at least $1 - \frac{1}{n^2}$, the total number of times the algorithm compares x_i with pivots is at most $d \lg n = 3(2 + c) \lg n = 3\left(2 + \log_{4/3} 2\right) \lg n$. The required value of d is $3\left(2 + \log_{4/3} 2\right) \leq 14$.

c Using a union bound for all the n elements of the original array A, we get that, with probability at least $1 - n\left(\frac{1}{n^2}\right) = 1 - \frac{1}{n}$, every value in the array is compared with pivots at most $d \lg n$ times, with d as in part (b). Therefore, with probability at least $1 - \frac{1}{n}$, the total number of such comparisons is at most $dn \lg n$. Using $d' = d$ works fine. Since all the comparisons made during execution of QUICKSORT involve comparison of some element with a pivot, we get the same probabilistic bound for the total number of comparisons.

d The modifications are easy. The Claim and part (a) are unchanged. For part (b), we now prove that with probability at least $1 - \frac{1}{n^{\alpha+1}}$, the total number of times the algorithm compares x_i with pivots is at most $d \lg n$, for $d = 3(\alpha + c)$. The argument is the same as before, but we use the

Claim with the value of α instead of 2 . Then for part (c), we show that with probability at least $1 - \frac{1}{n^\alpha}$, the total number of times the algorithm compares any value with a pivot is at most $dn \lg n$, where $d = 3(\alpha + c)$.

Reading materials

- *Introduction to Algorithms, Third Edition. Charpter 5 Probabilistic Analysis and Randomized Algorithms*
- *Introduction to Algorithms, Third Edition. Charpter 35 Approximation Algorithms*

External Exercise and Solution

1. Simulated annealing: https://en.wikipedia.org/wiki/Simulated_annealing
2. Code problem: https://www.luogu.com.cn/problem/P1337
3. Code problem: https://www.luogu.com.cn/problem/P2503